Loops in java

1. Introduction to Loops

Loops are control flow statements used to execute a block of code repeatedly until a given condition becomes false.

👉 Loops are essential for:

- Repeating tasks
- Reducing code duplication
- Automating logic
- Processing large data

---

2. Core Idea of Loop

A loop works on the principle of repetition controlled by a condition.

A loop continues as long as:
Condition = TRUE

Stops when:
Condition = FALSE

---

3. Structure of Loop

Every loop contains three important parts:

🔹 **Initialization**

Starting point of loop

int i = 0;

Condition

Defines how long loop runs

i < 5;

Update

Changes loop variable

i++;

---

4. Loop Execution Cycle

1. Initialization
2. Condition check
3. Execute loop body
4. Update variable
5. Repeat

---

5. Types of Loops

Java provides three main loops:

- for loop
- while loop
- do-while loop

---

6. for Loop

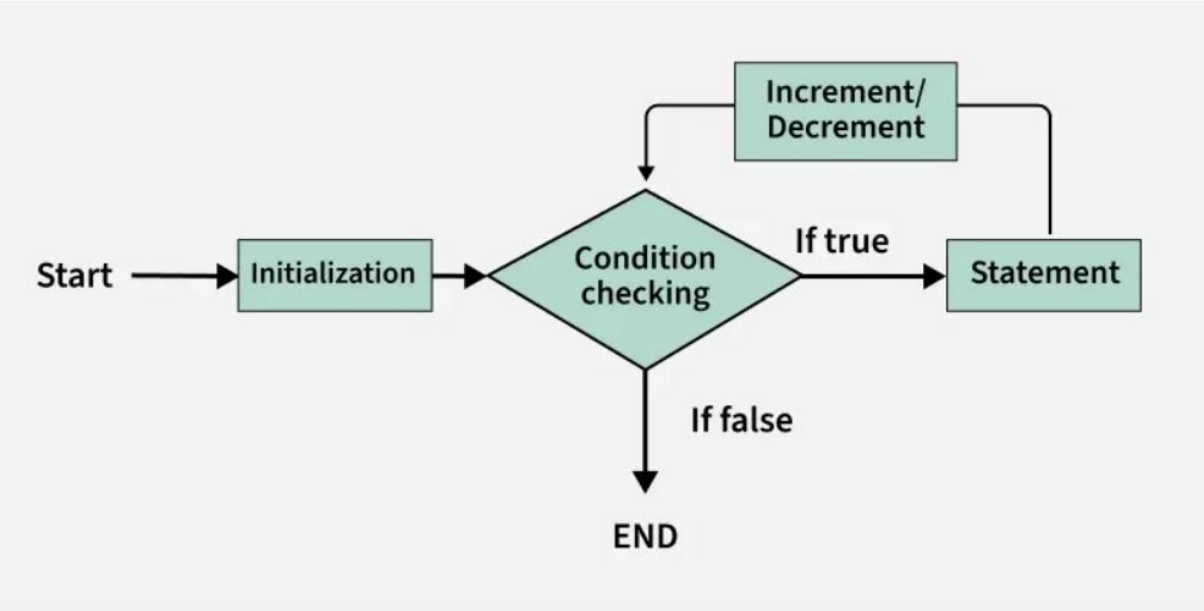
The for loop is used when we know the number of iterations (we know how many times we want to repeat a task). The for statement includes the initialization, condition, and increment/decrement in one line.

Example: The below Java program demonstrates a for loop that prints numbers from 0 to 10 in a single line.

```
// Java program to demonstrates the working of for loop

import java.io.*;


class Test {

  public static void main(String[] args)

  {

    for (int i = 0; i <= 10; i++) {

      System.out.print(i + " ");

    }

  }

}
```

Output :

0 1 2 3 4 5 6 7 8 9 10



- **Initialization condition**: Here, we initialize the variable in use. It marks the start of a for loop. An already declared variable can be used or a variable can be declared, local to loop only.
- **Testing Condition:** It is used for testing the exit condition for a loop. It must return a boolean value. It is also an Entry Control Loop as the condition is checked prior to the execution of the loop statements.
- **Statement execution**: Once the condition is evaluated to true, the statements in the loop body are executed.
- **Increment/ Decrement**: It is used for updating the variable for next iteration.
- **Loop termination**: When the condition becomes false, the loop terminates marking the end of its life cycle.

Note: There is another form of the for loop known as Enhanced for loop or (for each loop).

Enhanced for loop (for each)

This loop is used to iterate over arrays or collections.

Example: The below Java program demonstrates an Enhanced for loop (for each loop) to iterate through an array and print names.

```
// Java program to demonstrate
// the working of for each loop
import java.io.*;

class Geeks {
  public static void main(String[] args)
  {
    String[] names = { "Sweta", "Gudly", "Amiya" };

    for (String name : names) {
      System.out.println("Name: " + name);
    }
  }
}
```

Output

Name: Sweta

Name: Gudly

Name: Amiya

2. while Loop

A while loop is used when we want to check the condition before executing the loop body.

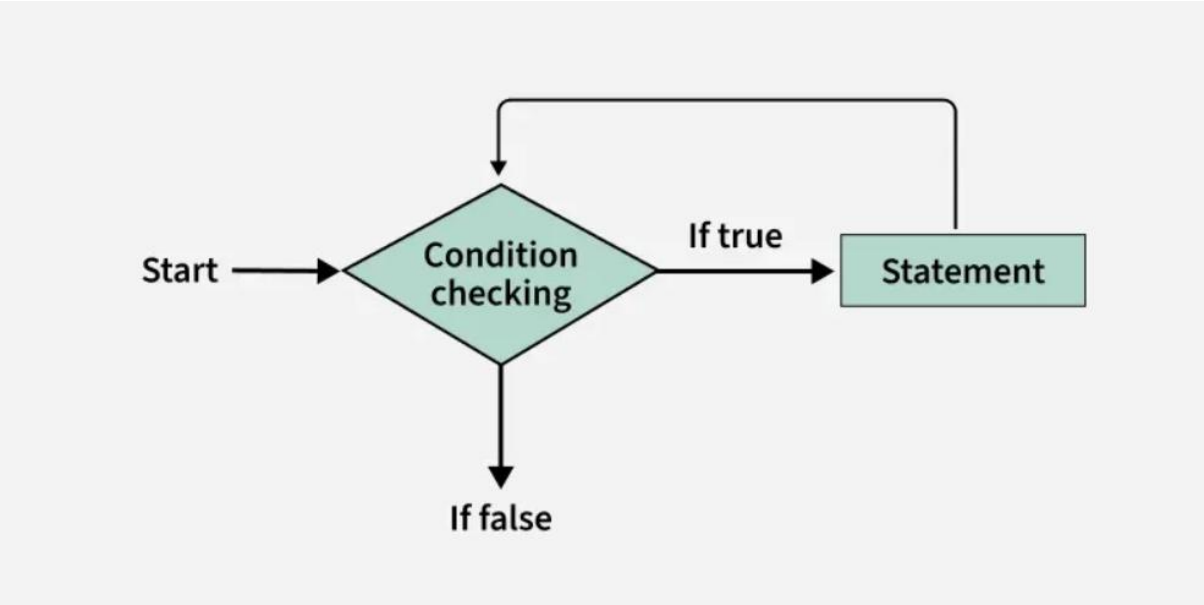
Example: The below Java program demonstrates a while loop that prints numbers from 0 to 10 in a single line.

```
// Java program to demonstrates
// the working of while loop


import java.io.*;
class Test {
  public static void main(String[] args)
  {
    int i = 0;
    while (i <= 10) {
      System.out.print(i + " ");
      i++;
    }
  }
}
```

Output

0 1 2 3 4 5 6 7 8 9 10

The below image demonstrates the flow chart of a while loop:



- While loop starts with the checking of Boolean condition. If it evaluated to true, then the loop body statements are executed otherwise first statement following the loop is executed. For this reason it is also called Entry control loop
- Once the condition is evaluated to true, the statements in the loop body are executed. Normally the statements contain an update value for the variable being processed for the next iteration.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.

3. do-while Loop

The do-while loop ensures that the code block executes at least once before checking the condition.

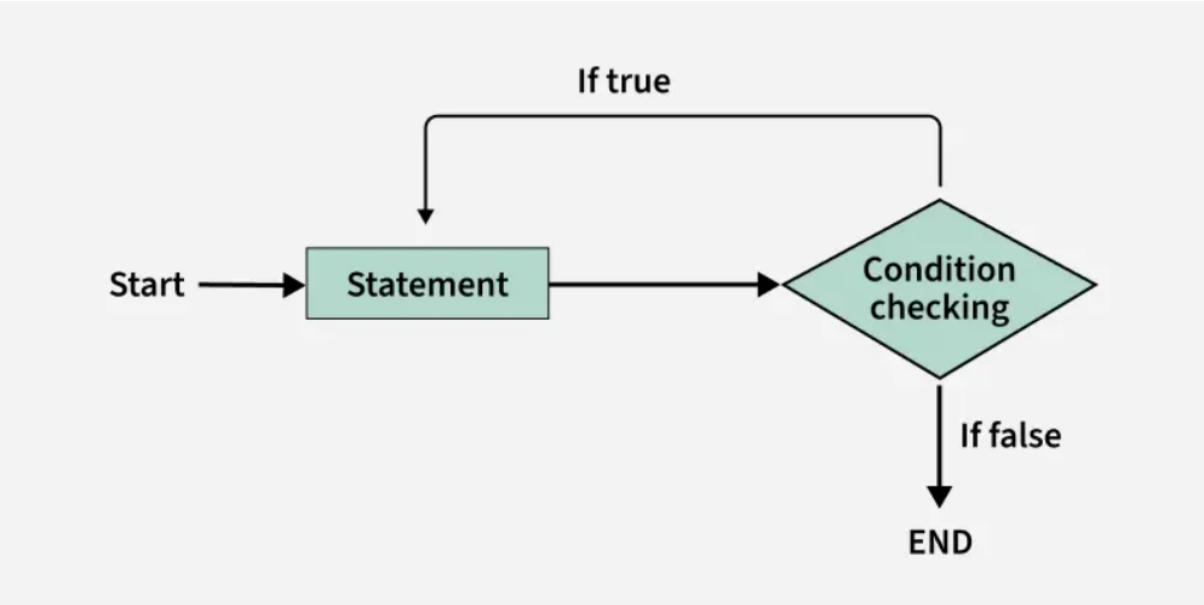
Example: The below Java program demonstrates a do-while loop that prints numbers from 0 to 10 in a single line.

```
// Java program to demonstrates
// the working of do-while loop
import java.io.*;

class Geeks {
  public static void main(String[] args)
  {
    int i = 0;
    do {
      System.out.print(i + " ");
      i++;
    } while (i <= 10);
  }
}
```

Output

0 1 2 3 4 5 6 7 8 9 10

The below image demonstrates the flow chart of a do-while loop:



- do while loop starts with the execution of the statement. There is no checking of any condition for the first time.
- After the execution of the statements, and update of the variable value, the condition is checked for true or false value. If it is evaluated to true, next iteration of loop starts.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.
- It is important to note that the do-while loop will execute its statements a tleast once before any condition is checked, and therefore is an example of exit control loop.

Common Loop Mistakes and How to Avoid them

If loops are not used correctly, they can introduce pitfalls and bugs that affect code performance, readability, and functionality. Below are some common pitfalls of loops:

1. Infinite Loops

This is one of the most common mistakes while implementing any sort of looping is that it may not ever exit, that is the loop runs for infinite time. This happens when the condition fails for some reason.

Types of Infinite Loops:

- infinite for Loop
- infinite while Loop

Example: Here, both the examples demonstrates the infinite loops.

```
// Java program to demonstrate
// the infinite for loop


import java.io.*;
class Test {
   public static void main(String[] args)
   {
      for (int i = 0; i < 5; i--) {
  System.out.println(
            "This loop will run forever");
      }
   }
}
```

Output: When you run both the above codes you will get TLE error.

2. Off-by-One Errors

Off-by-One Errors are caused when the loop runs one more or one fewer time than you wanted. It basically happens when the loop condition is not set correctly.

Example: The below Java program demonstrates an Off-by-One Error, where the loop runs 6 times and we expected it to run 5 times.

```
// Java Program to demonstrates Off-by-One Errors

import java.io.*;


class Test {

  public static void main(String[] args)

  {

    for (int i = 0; i <= 5; i++) {

      System.out.print(i + " ");

    }

  }

}
```

3. Modifying Loop Variables Inside the Loop

When we change the loop condition (like i) inside the loop, it can cause the loop to skip certain iterations or behave in ways that we did not expected. This might leads to errors or unexpected behavior.

Example: The below Java program demonstrates modifying the loop variable inside the loop, which cause the loop to skip certain iterations and behave unexpected.

```
// Java program demonstrates
// modification in i variable

import java.io.*;
class Test {
  public static void main(String[] args)
  {
    for (int i = 0; i < 5; i++) {
      if (i == 2) {

        // Modifies the loop variable and skips
        // the next iteration
        i++;
      }
      System.out.println(i);
    }
  }
}
```

4. Empty Loop Body

An empty loop body occurs when a loop is written to iterate but does not perform any operations inside the loop. Running a loop without any useful operations inside it can be confusing.

Example: The below Java program demonstrate Empty loop body.

```
// Java program to demonstrates Empty loop body
import java.io.*;

class Geeks {
  public static void main(String[] args)
  {
    for (int i = 0; i < 10; i++) {

      // Empty body no operations
    }
  }
}
```

Summary Table

Loop Type	When to Use	Condition Checking	Executes At Least Once?
for loop	When you want exact iterations	Before loop body, It is called Entry-controlled.	no
while loop	When you need condition check first.	Before loop body, It is called Entry-controlled.	no
do-while loop	When you need to run at least once	After loop body, It is called Exit-controlled.	yes
for-each loop	When you process all collection items	Internally handled	no

Nested Loop

A nested for loop is a loop inside another loop. It is commonly used for matrix operations and pattern printing.

Example: Printing a Matrix-like Pattern

```
class Geeks {
public static void main(String[] args) {
for (int i = 1; i <= 3; i++) {
for (int j = 1; j <= 3; j++) {
System.out.print("(" + i + "," + j + ") ");
}
System.out.println();
}
}
}
```

Output

(1,1) (1,2) (1,3)

(2,1) (2,2) (2,3)

(3,1) (3,2) (3,3)